

[Home](#) / [Field Guides](#) / [Agents](#) Copy as Markdown

How to match the latency of the homepage agent

Describes how we achieved the latency of our homepage agent.

Last Updated: 2026-02-20



Darryn Campbell

REFERENCE

From the [LiveKit homepage](#), select “Talk to LiveKit Agent” to speak with a voice agent about building with LiveKit. Many people assume we take special steps to reduce the latency of our homepage agent such as custom infrastructure, prioritized routing, or hidden optimizations. That’s not the case. The homepage agent is built using the same public features available to you, and you can create a similar low-latency experience for your own users.

There are actually multiple homepage agents, each with slightly different configurations, and you can transfer between them. If you look at the panels to the right of the running agent, you’ll see:

- The specific STT, LLM, and TTS models being used
- Real-time latency metrics for the current session

The specifics in this document are correct at the time of writing. Since our homepage agents are updated frequently, exact configurations may change in the future.

What all homepage agents have in common:

Deployed on LiveKit Cloud (us-east)

Our homepage agents are deployed on LiveKit Cloud in the us-east region.

The project hosting the agent is not subject to cold starts. To match the same startup time, you'll need to be on the **ship plan or higher**. Cold starts can significantly affect your initial start-up time, so eliminating them is important when optimizing for latency.

Latest version of Agents:

The homepage agent generally runs the latest version of LiveKit Agents. You should do the same to benefit from ongoing performance improvements and latency optimizations.

Although the homepage agent is written in Python, you can achieve similar performance using the Node.js Agents framework.

Turn detection and VAD settings:

The homepage agent uses the same turn detection and VAD configuration as our starter agent ↗.

Be sure to **prewarm your VAD**, this avoids model load time during the first interaction, which helps reduce initial latency:

```
1 def prewarm(proc: JobProcess):
2     proc.userdata["vad"] = silero.VAD.load()
3     ...
4 server.setup_fnc = prewarm
```

Then configure your session to use the latest Turn Detection model:

```
1 session=AgentSession(
2     ...
3     turn_detection=MultilingualModel(),
4     vad=ctx.proc.userdata["vad"],
5     ...
6 )
```

Agent Session Configuration:

All turn detection settings use their default values, except:

- `min_endpointing_delay = 0.2` (default: 0.5)
- `false_interruption_timeout = 1.0` (default: 2.0)

These values are slightly reduced from the defaults.

In practice, these adjustments are unlikely to significantly impact perceived latency compared to the default settings.

Speech settings:

Preemptive generation is enabled:

```
1 preemptive_generation=True
```

Note that enabling preemptive generation does **not** guarantee lower latency, but it can help in certain conversational flows by beginning response generation earlier.

Agent-specific configuration

Each homepage agent uses specific models for STT, LLM, and TTS (or a realtime model). To see the configuration for the active agent, refer to the “**Agent Configuration**” panel in the UI.

All homepage agents use **LiveKit Inference**. To match a specific agent, select the same models shown in the configuration panel.

For example, the agent **Hayley** uses:

- STT model: `Deepgram Nova-3`
- LLM model: `OpenAI GPT-4.1-mini` .
- TTS model: `Cartesia Sonic-3` .

This corresponds to:

```
1 stt="deepgram/nova-3",  
2 llm="openai/gpt-5.3-chat-latest",
```

```
3 tts="cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",  
4 # Note: This is the same tts voice as is used in the docs examples
```

None of the homepage agents use `extra_kwargs` to customize inference behavior.

User Interface

The agent communicates with the web front end through RPC. RPC is used to:

- Transfer between agents
- Report latency statistics.

The visual components come from the Agents UI package. Specifically, the homepage uses:

- AgentAudioVisualizerGrid.

Importantly, the front end does **not** affect agent latency.

You can use any of our front end starters to reproduce the experience, including:

- <https://github.com/livekit-examples/agent-starter-react> ↗
- The Agent Console.

Other Considerations

Latency statistics shown in the UI are sourced from session metrics, with overall conversation latency calculated as described in the docs.

Each homepage agent is initialized with:

- Personality instructions
- A high-level understanding of LiveKit

The homepage agent is not designed to provide comprehensive developer support. For technical assistance, please use our MCP server.

Conclusion

The homepage agent isn't powered by hidden optimizations or internal-only infrastructure. It uses the same public LiveKit Cloud deployment options, Agents framework, inference models, and configuration settings that are available to you.

If you want to match its latency:

- Deploy on LiveKit Cloud (being on the `ship` plan or higher to avoid cold starts)
- Use the latest Agents version
- Prewarm your VAD
- Match the same STT, LLM, and TTS models shown in the Agent Configuration panel

Low latency isn't the result of a single setting, it's the cumulative effect of good deployment choices, warm infrastructure, and appropriate model selection.

With the right setup, you can deliver the same responsive, real-time voice experience to your own users.

Was this helpful?



Read related documentation

Agents overview



Get started with voice AI agents

Quickstart guide



Build your first agent

Agent models



STT, TTS, and LLM plugins

Find more Agents guides

Building multi-agent architectures with LiveKit agents

Can you increase agent deployment limits?

Learn best practices for building multi-agent architectures including session state management, chat context handling,...

[View all Agents guides →](#)

Understand the hard cap on agent deployments and how to build a multi-tenant agent that scales without provisioning mor...



The open source framework and cloud platform for voice, video, and physical AI agents.

KEEP IN TOUCH



PRODUCTS

- Agent platform
- Media server
- SDKs
- Cloud dashboard
- Security
- Terms of service

RESOURCES

- Documentation
- Community
- Coding agent support
- Brand assets
- Video codecs
- Codec bitrates
- WebRTC browser test
- Connection test

COMPANY

- About
- Blog
- Careers
- Open source

[Terms of Service](#) | [Cookie Policy](#) | [Privacy Policy](#) | [Security](#)

ALL SYSTEMS OPERATIONAL